

CHAPTER 2

PART 4

Singly Linked List Implementation

Key topics

- Single Linked List: Implementation
 - Non-Generic SingleLinkedList
 - Single linked list of objects of a specific type
 - Generic SingleLinkedList
 - Single linked list of objects of any Java class type

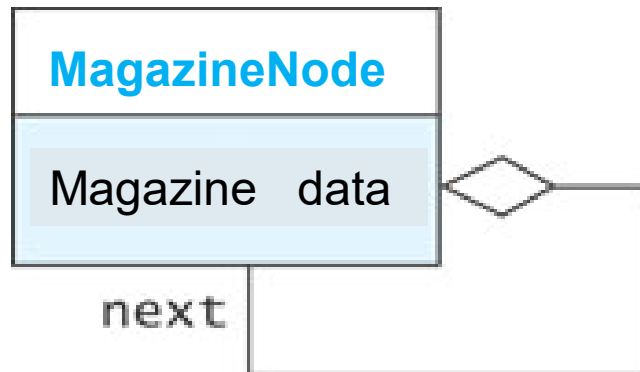
Implementing Single Linked List of objects of a Specific Class Type

- Do not use generics in implementation
- Node storing a reference to an object of a specific class type like Magazine (renamed to MagazineNode)
- Package linkedlist_1 objtype (FOR SELF-READING)
 - ▣ Magazine.java – Definition of all magazines
 - ▣ MagazineListSingleLinked.java –
 - Definition of single linked list of Magazine objects
 - Contains definition of a private MagazineNode class inside
 - ▣ MagazineListSingleLinkedTest.java
 - A Java application that uses class MagazineListSingleLinked.

Structure in a MagazineNode object for MagazineSingleLinkedList class

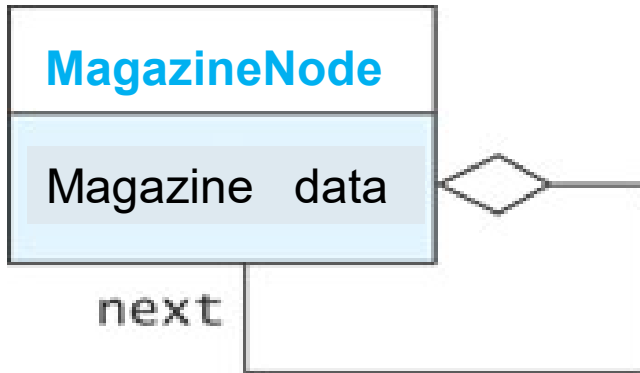
- A **MagazineNode** object in a single linked list of Magazine objects contains:
 - ▣ a data item of **Magazine** type
 - ▣ one **link** referencing the MagazineNode immediately after

MagazineNode structure:

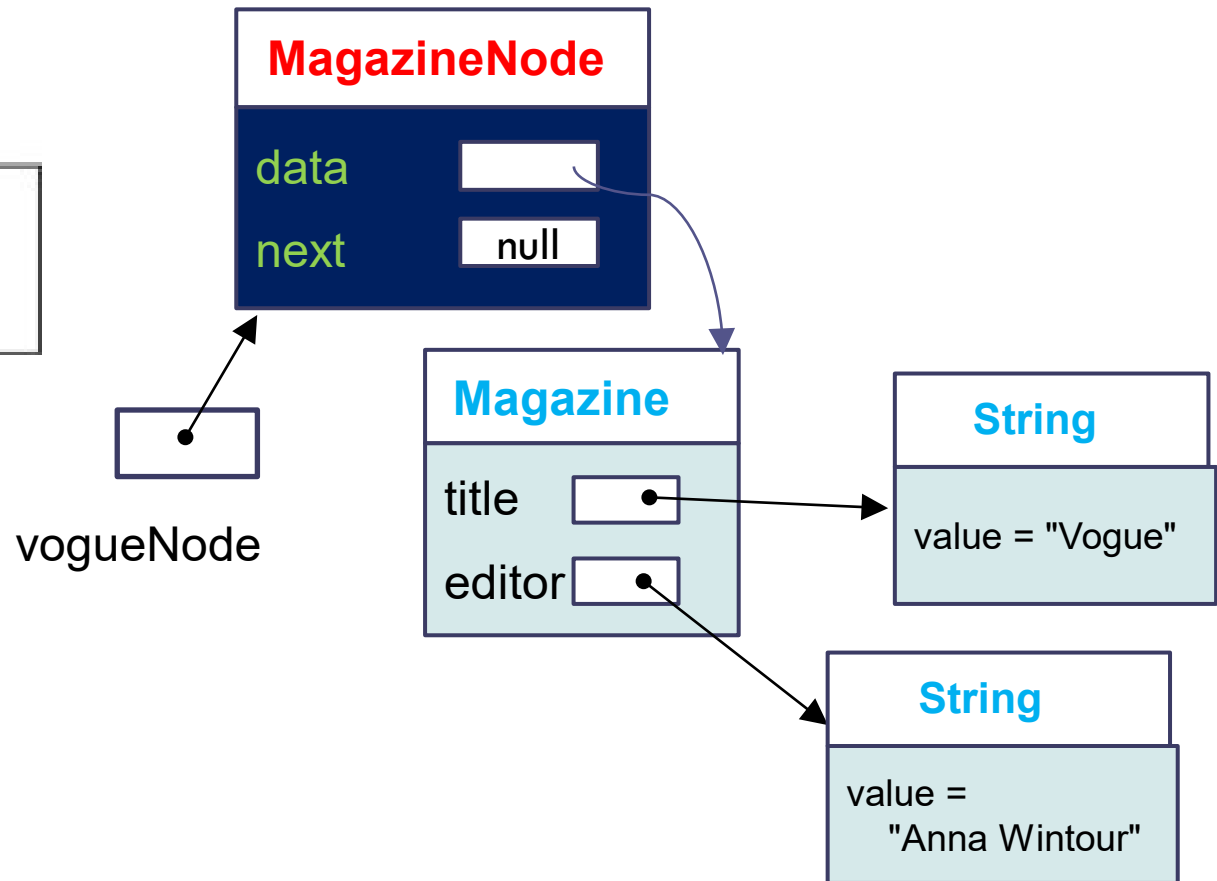


Structure in a MagazineNode object for MagazineSingleLinkedList class

MagazineNode structure:

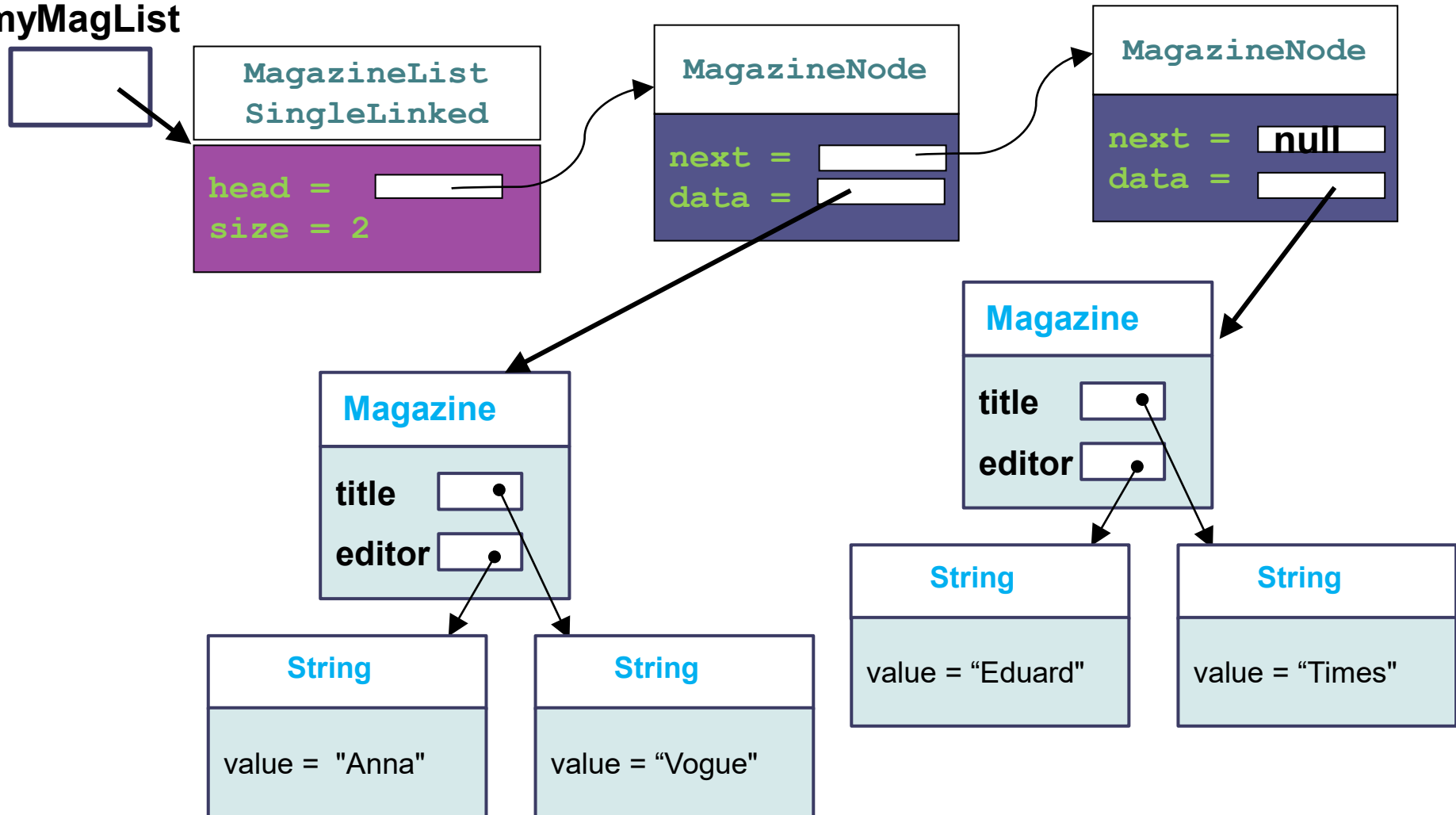


Sample MagazineNode object



MagazineListSingleLinked - Example

myMagList

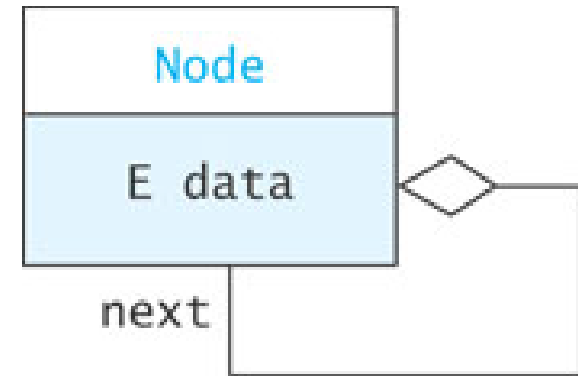


Implementing Generic Single Linked Lists

- Use generics in the implementation
- Node stores a reference to an object of any Java class type
 - ▣ Use type parameter `E`, representing `Student`, `String`, `Magazine`, etc
- Mark classes as generic: `Node<E>`, `SingleLinkedList<E>`
- Package `linkedlist_generic`
 - ▣ `SingleLinkedList.java`
 - Definition of single linked list of any Java objects
 - Contains definition of a `private Node<E>` class inside
 - ▣ `SingleLinkedListTest.java`
 - A Java application that uses class `SingleLinkedList<E>`

A Generic Single Linked List Node

- A **node** can contain:
 - ▣ a **data item**
 - ▣ one link
- A **link** is a **reference to a list node**
- In our structure, the node contains
 - ▣ a data field named `data` of type `E`
 - `E` – generic element type
 - `E` can be `Integer`, `String`, `Student` etc
 - ▣ and a reference to the next node, named `next`



Directly Connecting Nodes:

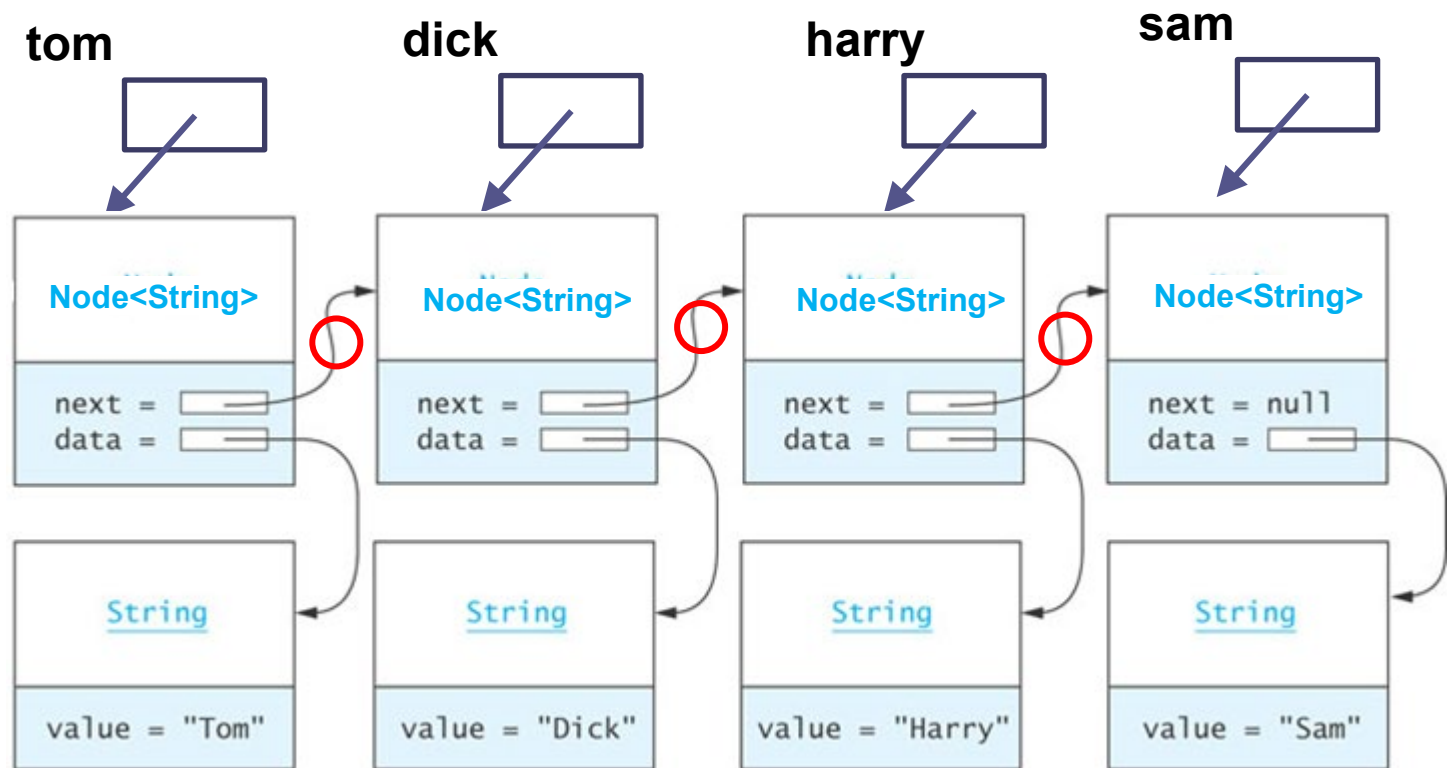
generic Node<E>

- Connect nodes **DIRECTLY INSIDE** class SingleLinkedList<E> because Node<E> class is private.
 - See main(...) method inside class SingleLinkedList<E>.java
 - They are similar to the code in file UseNodes.java.

```
Node<String> tom = new Node<>("Tom");  
//Or var tom = new Node<String>("Tom");  
Node<String> dick = new Node<>("Dick");  
Node<String> harry = new Node<>("Harry");  
Node<String> sam = new Node<>("Sam");  
  
//connect the Node<String> objects  
tom.next = dick;  
dick.next = harry;  
harry.next = sam;
```

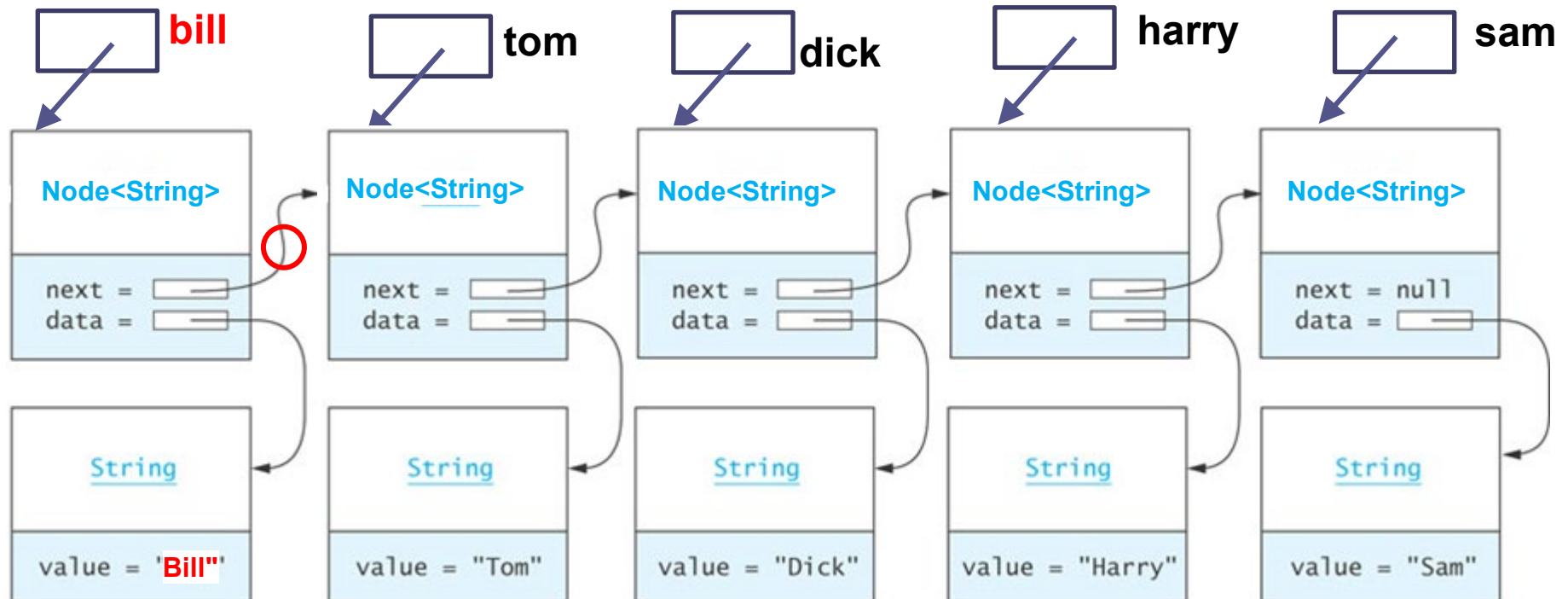
Directly Connecting Nodes:

generic Node<E> (cont'd)



The figure shows the resulting list from the previous slide.

Add First: generic Node<E>

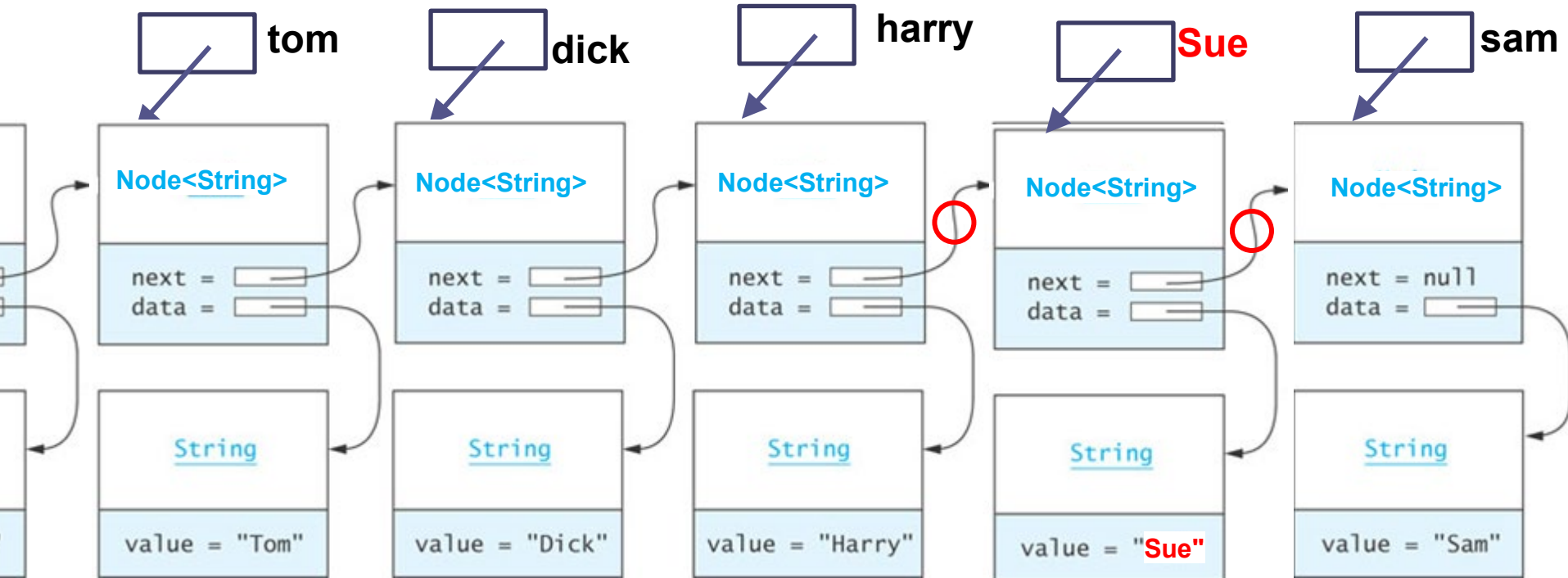


```
Node<String> bill = new Node<>("Bill");  
bill.next = tom;
```

OR

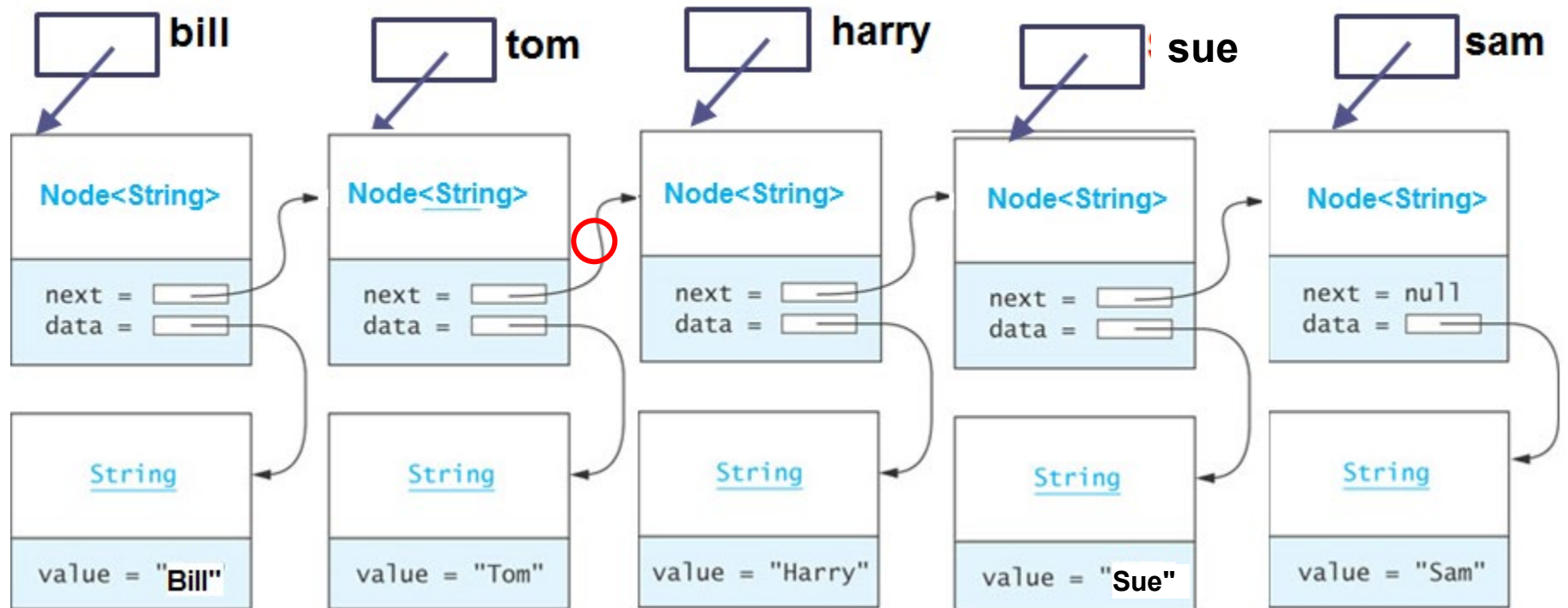
```
Node<String> bill = new Node<>("Bill", tom);
```

Add in-between: generic Node<E>



```
Node<String> sue = new Node<>("Sue", sam);  
harry.next = sue;
```

Remove Dick: generic Node<E>



```
tom.next = dick.next;
```

Or

```
tom.next = tom.next.next;
```

Indirectly Connecting Nodes:

generic Node<E>

- Connect nodes **INDIRECTLY OUTSIDE** class `SingleLinkedList<E>`
 - Program: `ConnectNodesGenericIndirect.java`

```
SingleLinkedList<String> names
    = new SingleLinkedList<>();
//connect nodes indirectly
names.add("Tom");
names.add("Dick");
names.add("Harry");
names.add("Sam");
```

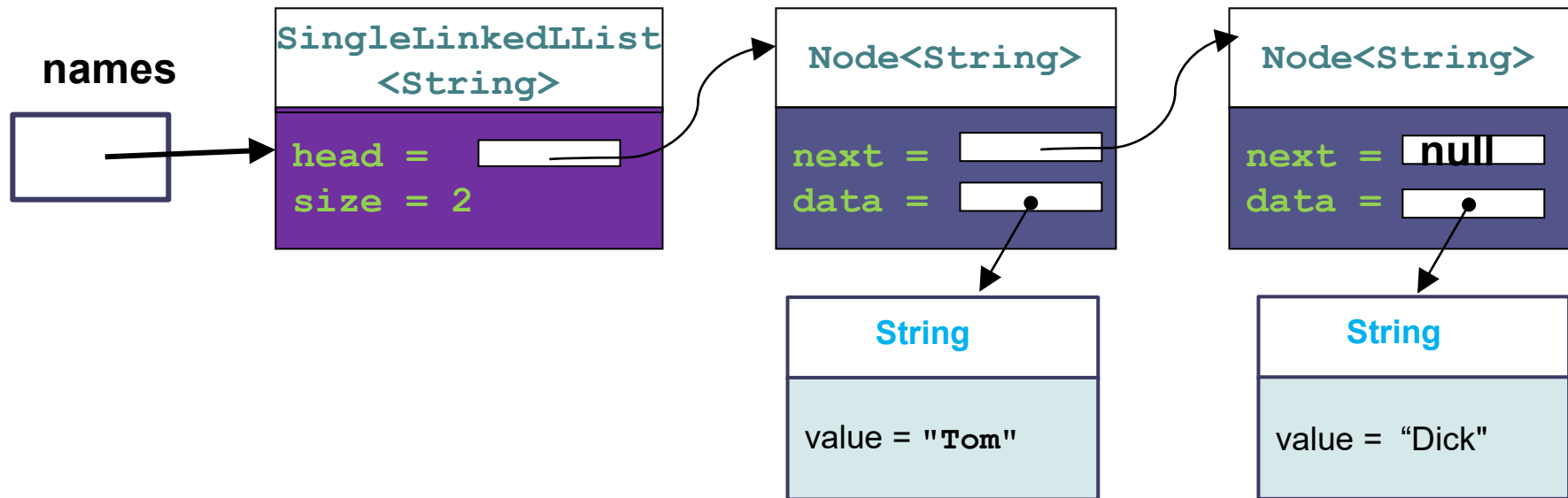
A Generic Single-Linked List Class

- Generally, we do not have individual references to each node.
- A `SingleLinkedList` object has a data field `head`, the *list head reference*, which references the first list node

```
public class SingleLinkedList<E> {  
    private Node<E> head = null;  
    private int size = 0;  
    ...  
}
```

Example of SingleLinkedList <E>:

SingleLinkedList<String> names



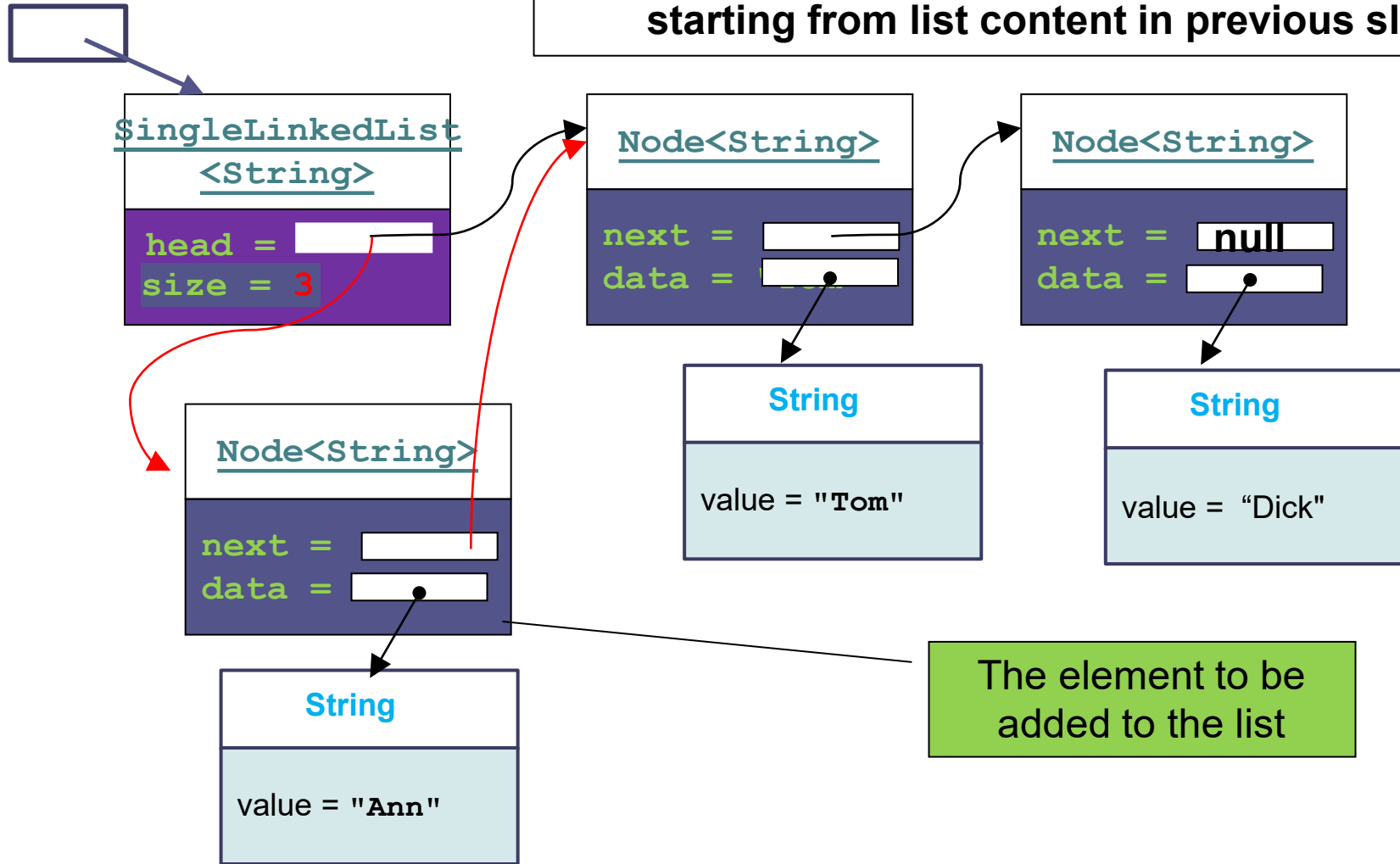
- names is a reference variable of type **SingleLinkedList<String>**. Its declaration:
`SingleLinkedList<String> names;`
- The figure above is the result of the following operations:
`names.add("Tom");`
`names.add("Dick");`

Implementing

```
public void addFirst(E item)
```

names

**Details of action: `names.addFirst("Ann")` ;
starting from list content in previous slide.**



Implementing

public void addFirst(E item) (cont.)

```
public void addFirst (E item) {  
    Node<E> temp = new Node<E>(item, head);  
    head = temp;  
    size++;  
}
```

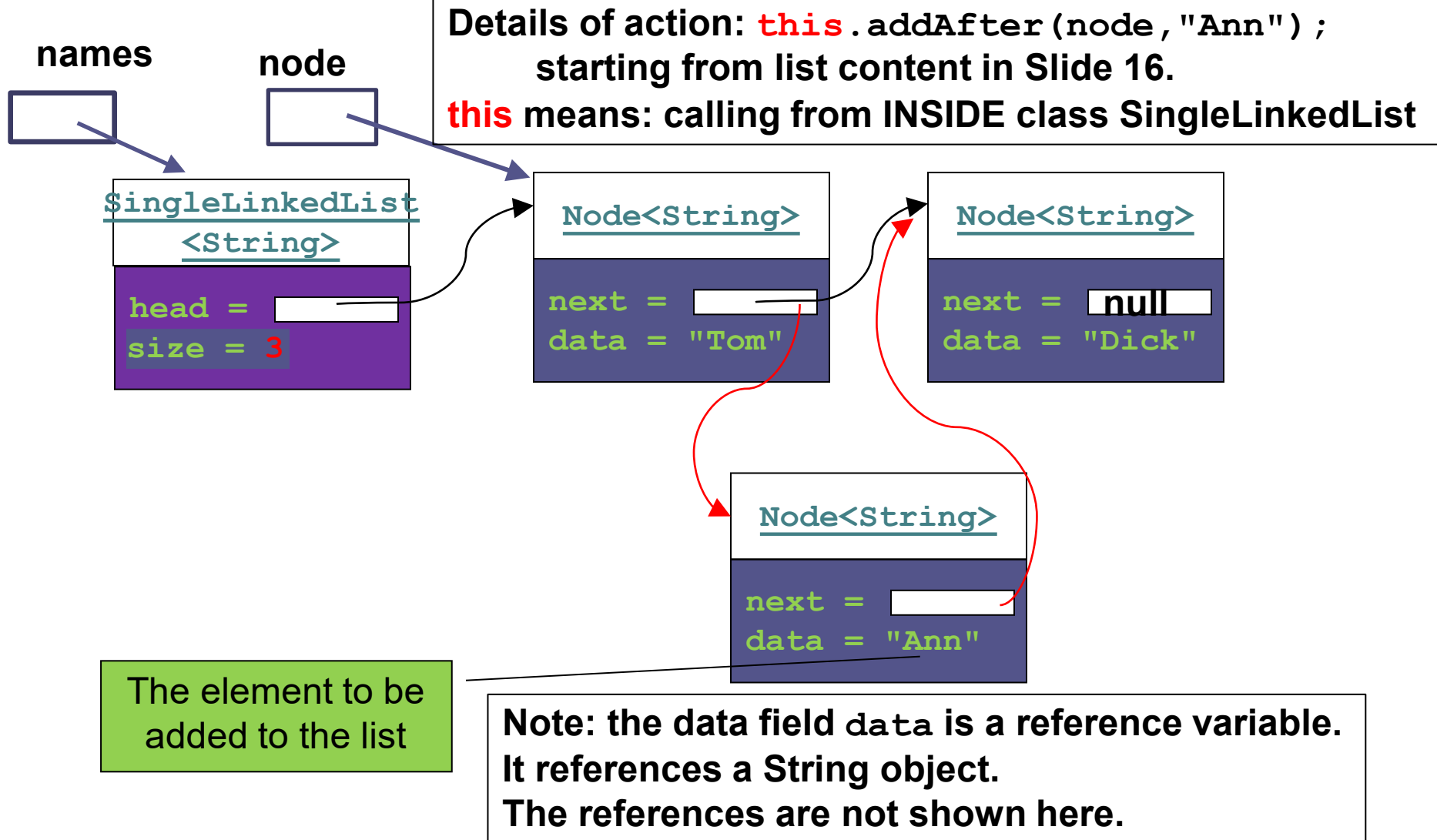
or, more simply ...

```
public void addFirst (E item) {  
    head = new Node<E>(item, head);  
    size++;  
}
```

This works even if head is null

Implementing helper method

```
private void addAfter(Node<E> node, E item)
```



Implementing helper method (cont.)

```
private void addAfter (Node<E> node, E item)
```

```
private void addAfter (Node<E> node, E item) {  
    Node<E> temp = new Node<E>(item, node.next);  
    node.next = temp;  
    size++;  
}
```

or, more simply ...

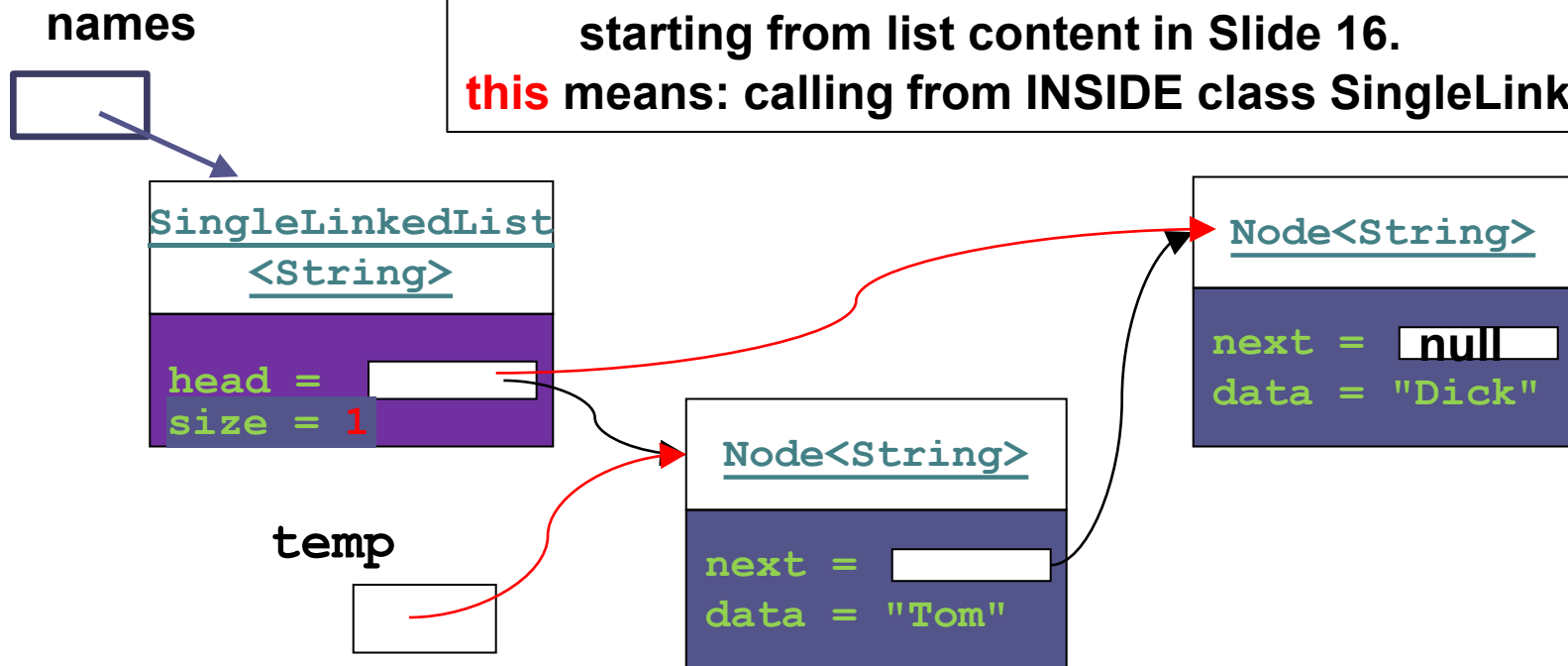
```
private void addAfter (Node<E> node, E item) {  
    node.next = new Node<E>(item, node.next);  
    size++;  
}
```

We declare this method `private` since it should not be called from outside the class. Later we will see how this method is used to implement the public add methods.

Implementing helper method

```
private E removeFirst()
```

Details of action: **this.removeFirst()** ;
starting from list content in Slide 16.
this means: calling from INSIDE class SingleLinkedList



Note: the data field data is a reference variable.
It references a String object.
The references are not shown here.

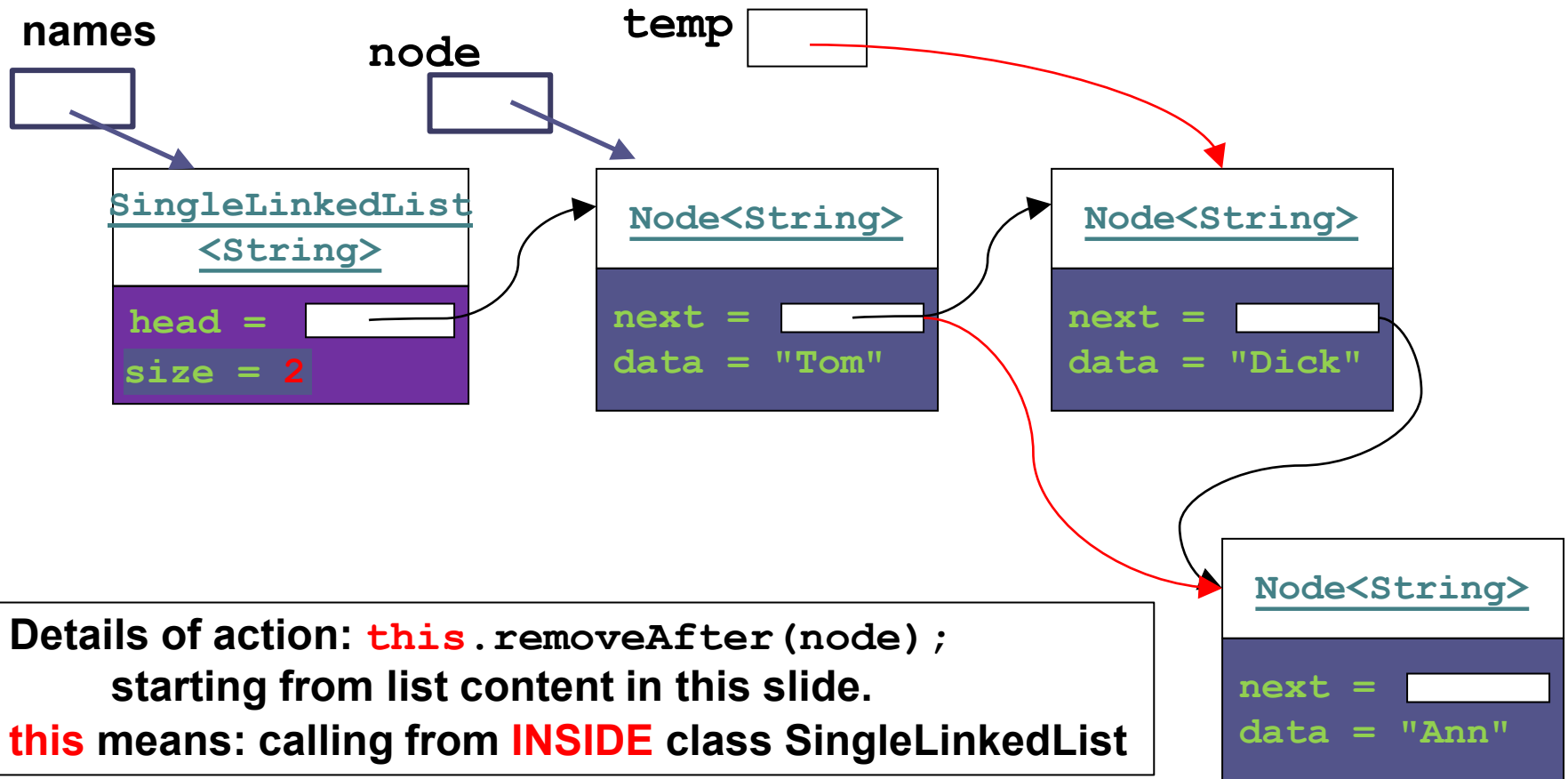
Implementing helper method

```
private E removeFirst() (cont.)
```

```
private E removeFirst () {  
    Node<E> temp = head;  
    if (head != null) {  
        head = head.next;  
    }  
    if (temp != null) {  
        size--;  
        return temp.data;  
    } else {  
        return null;  
    }  
}
```

Implementing helper method

```
private E removeAfter(Node<E> node)
```



Note: the data field `data` is a reference variable.
It references a `String` object.
The references are not shown here.

Implementing helper method (cont.)

```
private E removeAfter (Node<E> node)
```

```
private E removeAfter (Node<E> node) {  
    Node<E> temp = node.next;  
    if (temp != null) {  
        node.next = temp.next;  
        size--;  
        return temp.data;  
    } else {  
        return null;  
    }  
}
```


Implementing helper method

```
private Node<E> getNode(int index)
```

```
private Node<E> getNode(int index) {  
    Node<E> node = head;  
    for (int i=0; i<index && node != null; i++) {  
        node = node.next;  
    }  
    return node;  
}
```

Completing the SingleLinkedList<E> Class

Method	Behavior
<code>public E get(int index)</code>	Returns a reference to the element at position <code>index</code> .
<code>public E set(int index, E anEntry)</code>	Sets the element at position <code>index</code> to reference <code>anEntry</code> . Returns the previous value.
<code>public int size()</code>	Gets the current size of the List.
<code>public boolean add(E anEntry)</code>	Adds a reference to <code>anEntry</code> at the end of the List. Always returns <code>true</code> .
<code>public void add(int index, E anEntry)</code>	Adds a reference to <code>anEntry</code> , inserting it before the item at position <code>index</code> .
<code>public int indexOf(E target)</code>	Searches for <code>target</code> and returns the position of the first occurrence, or <code>-1</code> if it is not in the List.

Implementing

```
public E get(int index)
```

```
public E get (int index) {  
    if (index < 0 || index >= size) {  
        throw  
            new IndexOutOfBoundsException(Integer.toString(index));  
    }  
    Node<E> node = getNode(index);  
    return node.data;  
}
```

Implementing

```
public E set(int index, E newValue)
```

```
public E set (int index, E newValue) {  
    if (index < 0 || index >= size) {  
        throw new  
            IndexOutOfBoundsException(Integer.toString(index));  
    }  
    Node<E> node = getNode(index);  
    E result = node.data;  
    node.data = newValue;  
    return result;  
}
```

Implementing

```
public void add(int index, E item)
```

```
public void add (int index, E item) {  
    if (index < 0 || index > size) { //not index >= size  
        throw new  
            IndexOutOfBoundsException(Integer.toString(index));  
    }  
    if (index == 0) {  
        addFirst(item);  
    } else {  
        Node<E> node = getNode(index-1);  
        addAfter(node, item);  
    }  
}
```

Implementing

```
public boolean add(E item)
```

- To add an item to the end of the list

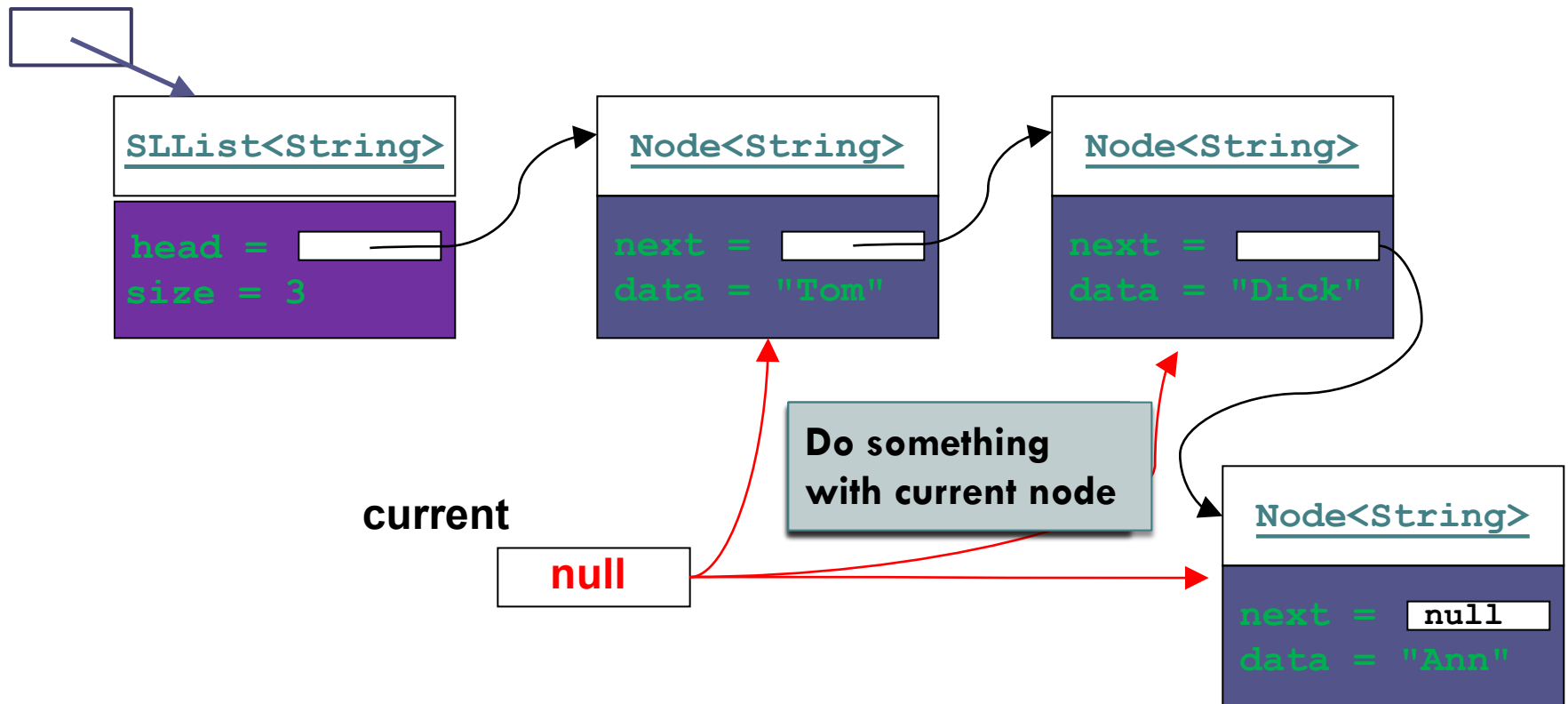
```
public boolean add (E item) {  
    add(size, item);  
    return true;  
}
```

Note:

The add calls the public method: `add(int index, E item)`, which allows index to be size, meaning appending to the end of the list.

Traversing a Generic Single-Linked List

names



Note: the data field `data` is a reference variable. It references a `String` object. The references are not shown here.

Traversing a generic Single-Linked List

(cont.)

- **toString() can be implemented with a traversal:**
- **Use StringBuilder to hold intermediate resulting strings.**

```
public String toString() {  
    StringBuilder listInfo = new StringBuilder("[");  
    Node<E> current = head;  
    while (current != null) {  
        listInfo.append(current.data.toString());  
        listInfo.append(", ");  
        current = current.next;  
    }  
    listInfo.append("]");  
    return listInfo.toString();  
}
```


Use **class** `SingleLinkedList<E>`

- Programs (Not in book)

- ▣ `SingleLinkedListTestString.java`

- Use `SingleLinkedList<E>` for creating and processing a single linked list of `String` objects (used for movie titles)

- ▣ `SingleLinkedListTestMagazine.java`

- Use `SingleLinkedList<E>` for creating and processing a single linked list of `Magazine` objects